Threaded BT/BST

Threaded Trees

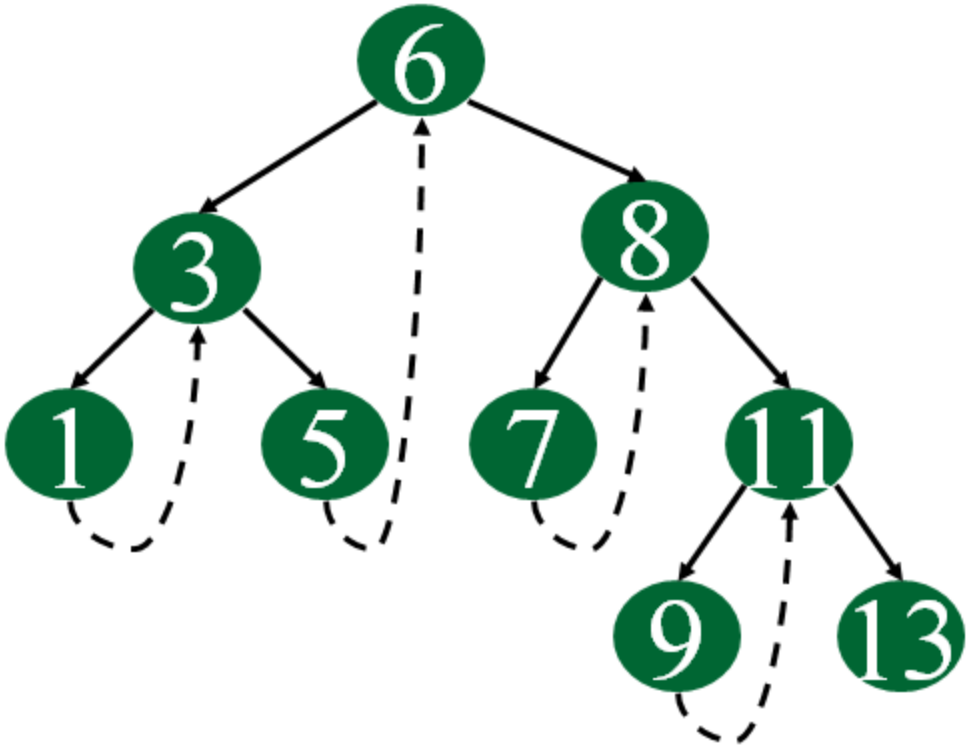
- Binary trees have a lot of wasted space: the leaf nodes each have 2 null pointers
- We can use these pointers to help us in inorder traversals
- We have the pointers reference the next node in an inorder traversal; called *threads*
- We need to know if a pointer is an actual link or a thread, so we keep a boolean for each pointer

- Inorder traversal of a Binary tree can either be done using recursion or with the use of a auxiliary stack. The idea of threaded binary trees is to make inorder traversal faster and do it without stack and without recursion. A binary tree is made threaded by making all right child pointers that would normally be NULL point to the inorder successor of the node (if it exists).
- The threads are also useful for fast accessing ancestors of a node.

There are two types of threaded binary trees.

- ***Single Threaded:*** Where a NULL right pointers is made to point to the inorder successor (if successor exists)
- ***Double Threaded:*** Where both left and right NULL pointers are made to point to inorder predecessor and inorder successor respectively. The predecessor threads are useful for reverse inorder traversal and postorder traversal.

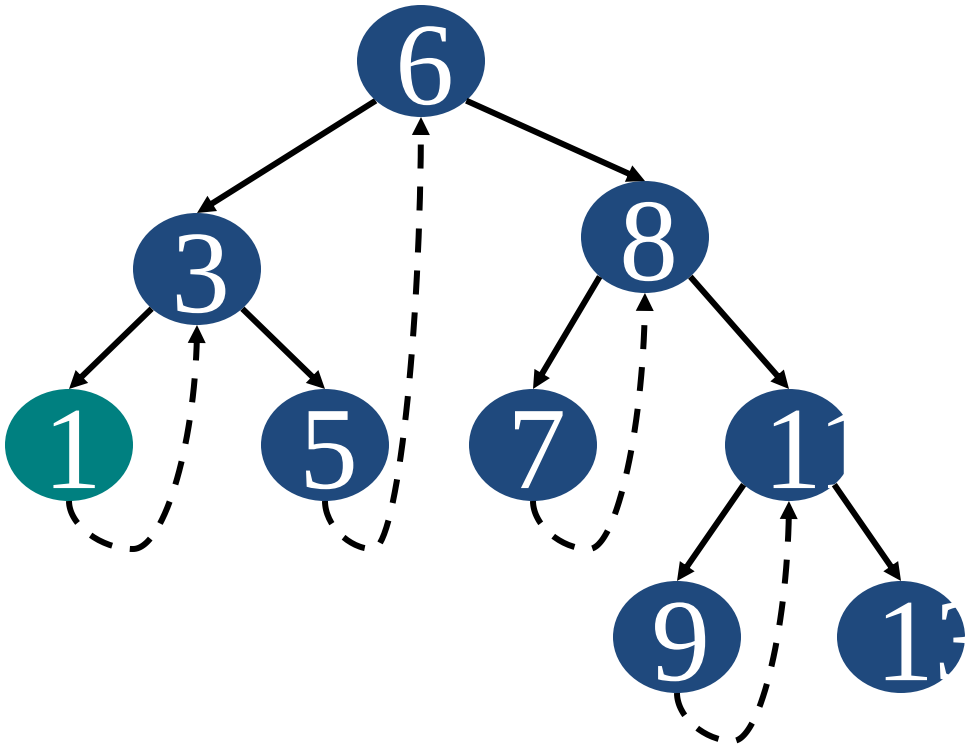
Threaded Tree



Threaded Tree Traversal

- We start at the leftmost node in the tree, print it, and follow its right thread
- If we follow a thread to the right, we output the node and continue to its right
- If we follow a link to the right, we go to the leftmost node, print it, and continue

Threaded Tree Traversal

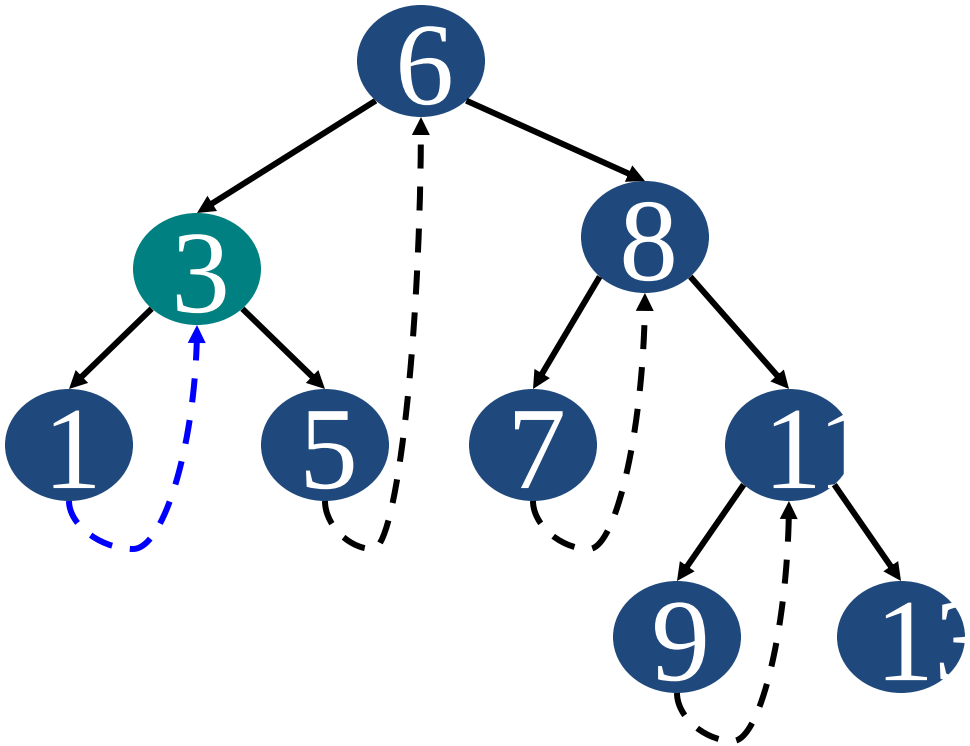


Output

1

Start at leftmost node, print it

Threaded Tree Traversal



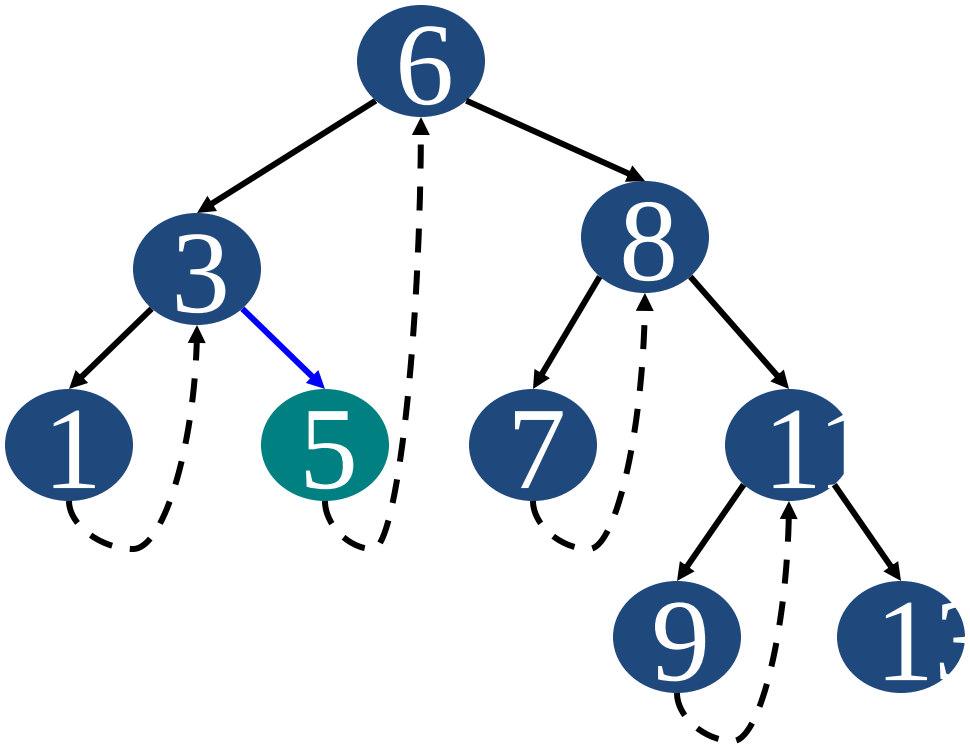
Output

1

3

Follow thread to right, print node

Threaded Tree Traversal



Output

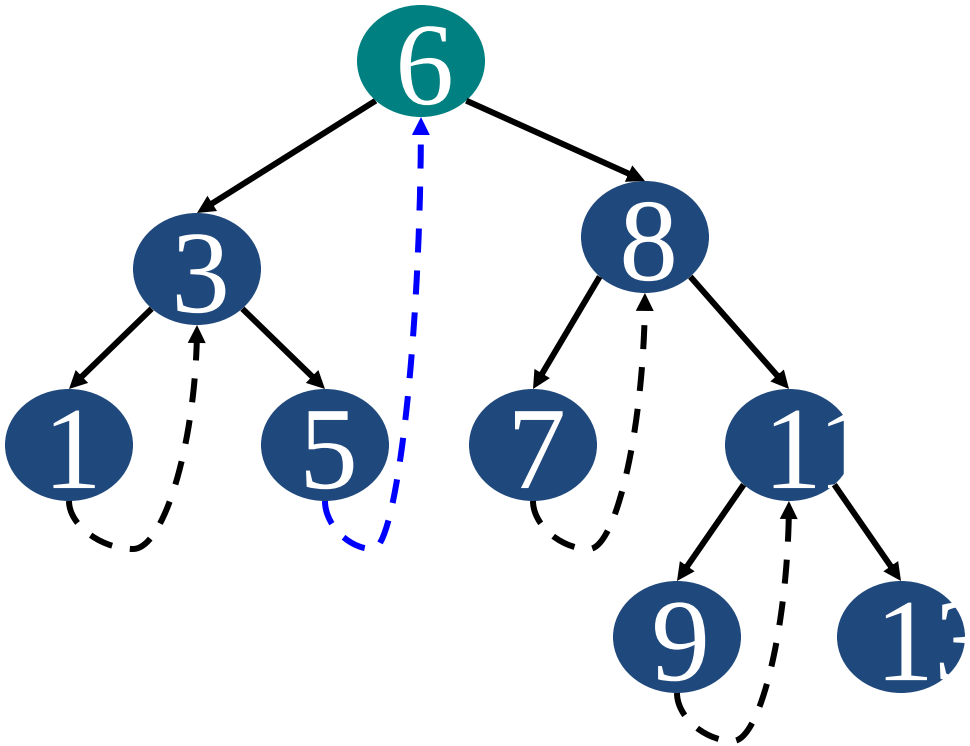
1

3

5

Follow link to right, go to leftmost node and print

Threaded Tree Traversal



Output

1

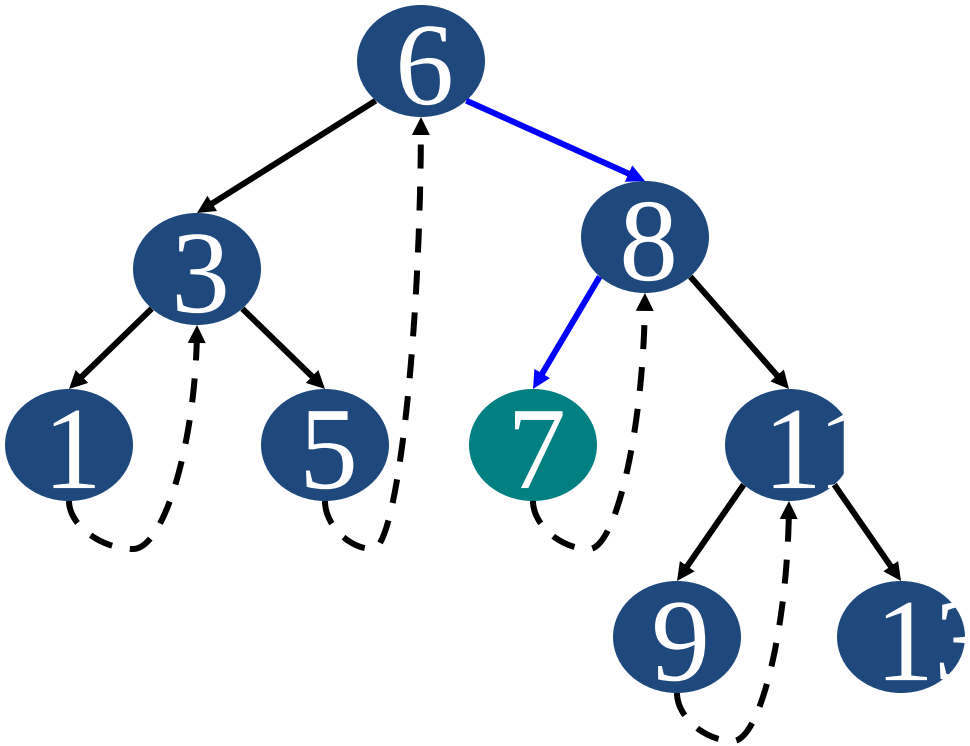
3

5

6

Follow thread to right, print node

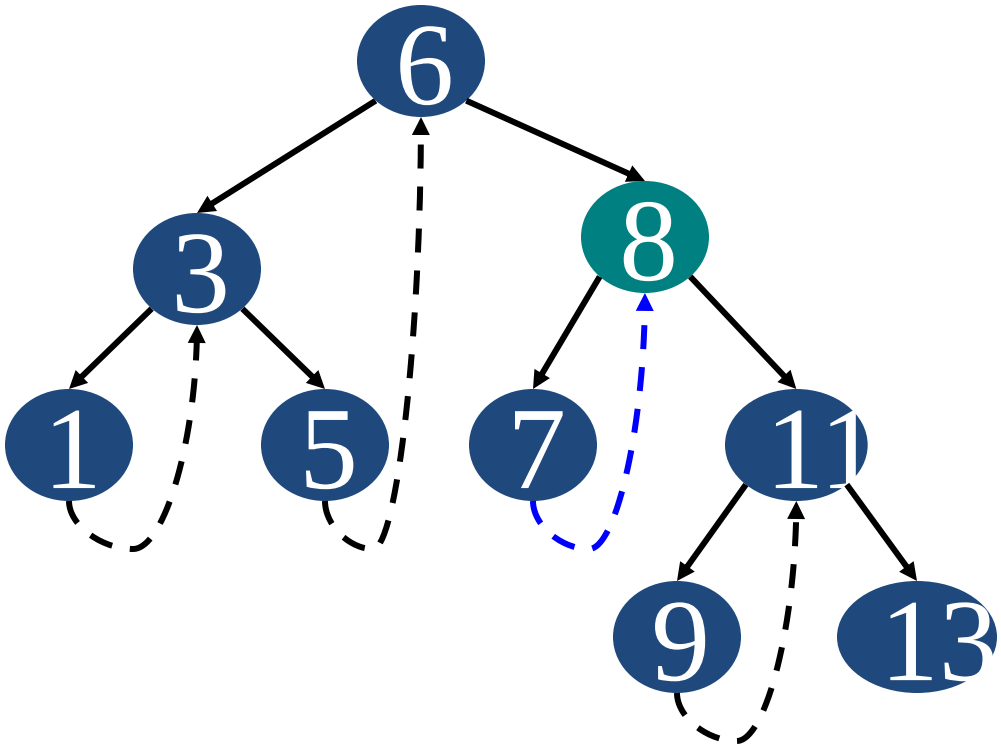
Threaded Tree Traversal



Output
1
3
5
6
7

Follow link to right, go to leftmost node and print

Threaded Tree Traversal

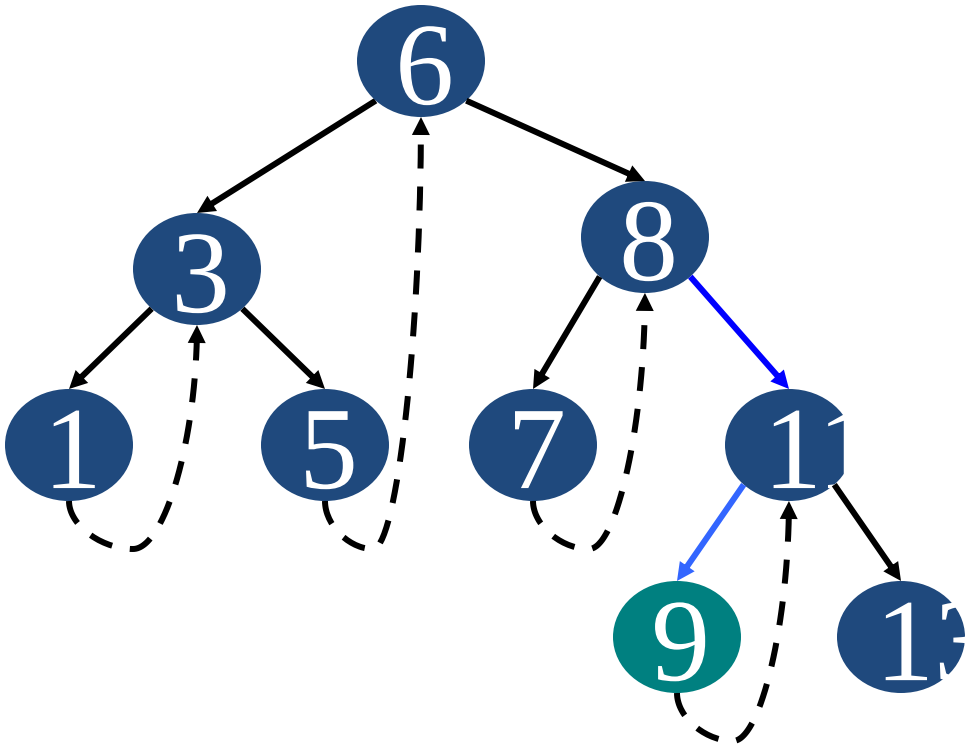


Output

1
3
5
6
7
8

Follow thread to right, print node

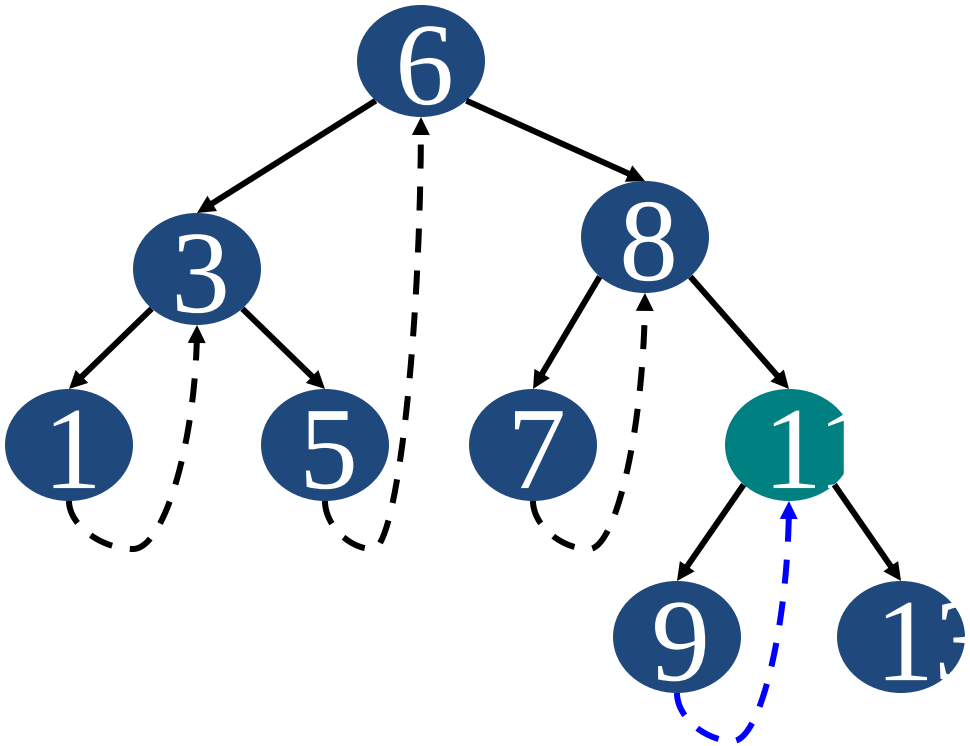
Threaded Tree Traversal



Output
1
3
5
6
7
8
9

Follow link to right, go to leftmost node and print

Threaded Tree Traversal

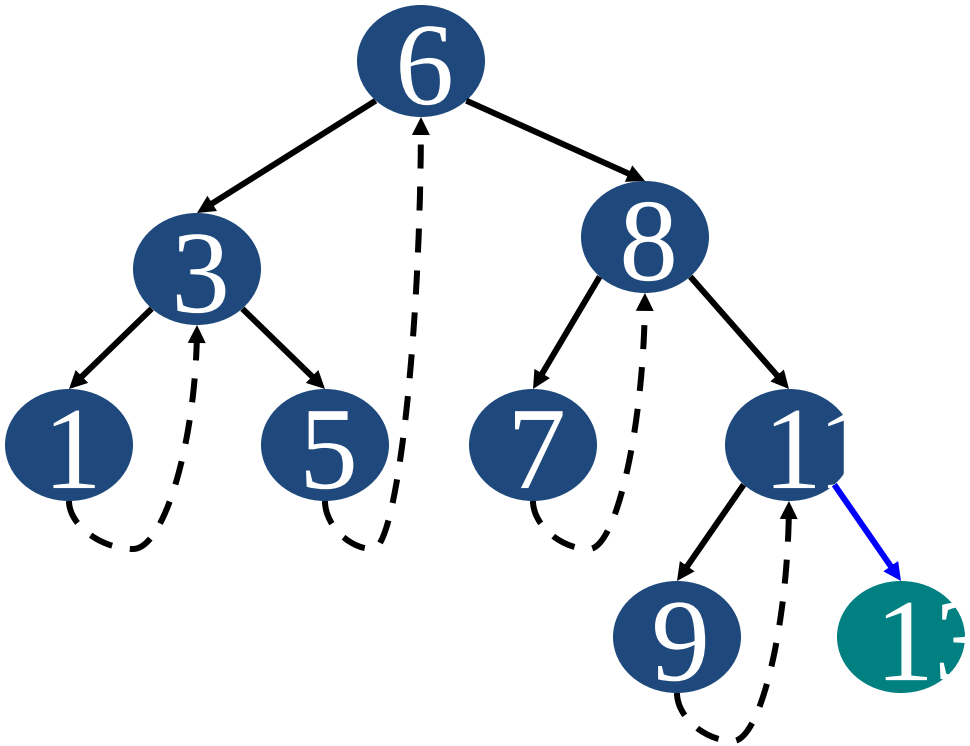


Output

1
3
5
6
7
8
9
11

Follow thread to right, print node

Threaded Tree Traversal



Output
1
3
5
6
7
8
9
11
13

Follow link to right, go to leftmost node and print

```
struct Node
{
    int data;
    Node *left, *right;
    bool rightThread;
}
```

Threaded Tree Traversal Code

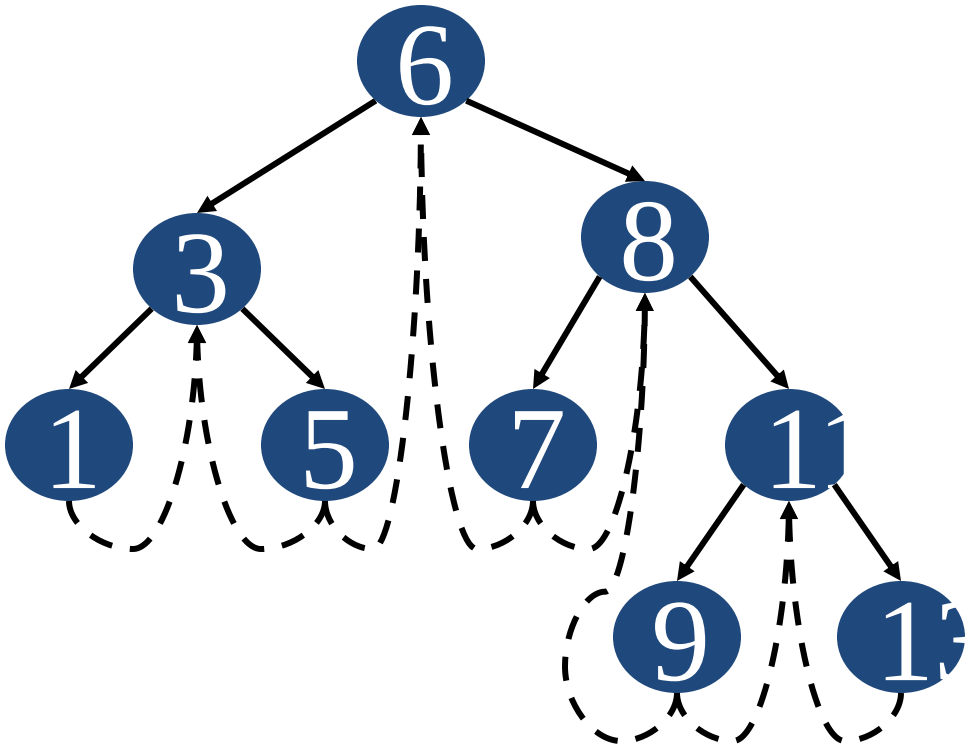
```
// C code to do inorder traversal in a threaded binary tree
void inOrder(struct Node *root)
{
   struct Node *cur = leftmost(root);
   while (cur != NULL)
   {
      printf("%d ", cur->data);

      // If this node is a thread node, then go to
      // inorder successor
      if (cur->rightThread)
         cur = cur->right;
      else // Else go to the leftmost child in right subtree
         cur = leftmost(cur->right);
   }
}
```

```
struct Node* leftMost(struct Node *n)
{
   if (n == NULL)
      return NULL;

   while (n->left != NULL)
       n = n->left;

   return n;
}
```

Threaded Tree Modification

- We're still wasting pointers, since half of our leafs' pointers are still null
- We can add threads to the previous node in an inorder traversal as well, which we can use to traverse the tree backwards or even to do postorder traversals

Threaded Tree Modification



Insertion

- First element
- Left Child
- Right Child

- **First element**

root = temp;

temp -> left = **NULL**;

temp -> right = **NULL**;

- **Left Child**

temp -> left = **parent** ->left;

temp -> right = **parent**;

parent -> leftThreaded = **false**;

parent -> left = temp;

- **Right Child**

temp -> left = **parent**;

temp -> right = **parent** -> right;

parent -> rightThreaded = **false**;

parent -> right = temp;

```
struct Node insert(struct Node root, int val)
{
// Search for the correct position
struct Node *ptr = root;
struct Node *parent = NULL; // parentent of key to be inserted
while (ptr != NULL)
{

    // Moving on left subtree.
    if (val < ptr->data)
    {
        if (ptr -> leftThreaded == false)
            ptr = ptr -> left;
        else
            break;
    }
    // Moving on right subtree.
    else
    {
        if (ptr->rightThreaded == false)
            ptr = ptr -> right;
        else
            break;
    }
}
```

```
// Create a new node
struct Node *temp = (struct Node* )malloc(sizeof(struct Node));
temp -> data = val;
temp -> leftThreaded = true;
temp -> rightThreaded = true;
if (parent == NULL)
{
    root = temp;
    temp -> left = NULL;
    temp -> right = NULL;
}
else if (val < (parent -> data))
{
    temp -> left = parent -> left;
    temp -> right = parent;
    parent -> leftThreaded = false;
    parent -> left = temp;
}
else
{
    temp -> left = parent;
    temp -> right = parent -> right;
    parent -> rightThreaded = false;
    parent -> right = temp;
}
return root;
}
```